

DEEP LEARNING FROM SCRATCH

A Guide for Self-Learners

Episode III

Birth of a Neuron

*AI evolves fast, but speed isn't the enemy: confusion is.
So let's go fast, in the right direction.*

Why this mini-guide?

Episode I revealed how a neuron thinks.

Episode II showed how it learns.

Episode III makes it real... in code.

How this mini-guide?

We give life to a neuron, coding by hand its intelligence.

"The most difficult thing is the decision to act; the rest is merely tenacity."

– Amelia Earhart

Pierre Chambet

November 2025

[Launch birth_of_a_neuron.ipynb on Google Colab](#)

Run all the code, visualize the learning process, and experiment freely.

Here, You Code Our First Artificial Neuron

Message for Non-Coders

Don't get lost in the code. You're not here to prove you're a Python wizard. You're here to understand the architecture - and how the pieces fit together.

Below, you'll see some weird stuff like:

- `numpy (np)` to handle arrays and vector operations
- `matplotlib (plt)` to plot data and learning curves
- `sklearn` to generate a toy dataset

Don't panic if you don't know what every line does. That's not the point. What matters is this:

You don't need to know how every tool works under the hood. You need to know what the tool does - and why you're using it.

That's enough to build a neuron. That's enough to build intelligence.
So keep that mindset and walk through the fire. **There will be a before and an after this notebook for you.**

Theory in Short (Reminder)

A neuron is simply a linear model with two learnable parameters: $\mathbf{W}$ and $\mathbf{b}$. Our goal is to find the best values for these parameters so that the neuron makes good predictions.

To do that, we loop through a learning procedure based on four essential steps:

1. **Model Function**

Defines how the neuron behaves - a linear transformation followed by an activation function (like sigmoid).

2. **Cost Function**

Measures the error between predicted outputs and ground-truth labels y . It quantifies how well the neuron performs.

3. **Gradients Function**

Computes the derivatives of the cost function with respect to the parameters - telling us in which direction to adjust them.

4. **Update Function**

Applies the gradients to adjust W and b slightly, using a learning rate α .

This is the generic learning architecture behind **every neural network** - from a single neuron to GPT-5. We will code that.

Let's build our first neuron

We're about to code our very first artificial neuron. You'll see: once the structure is in place, everything else becomes modular.

This is the real beginning of your deep learning journey. **No magic, no shortcuts**

*First question every data scientist asks is simple: **what is my dataset?***

Because behind every dataset, there is a goal, a purpose, someone you're trying to help.

Purpose is the beginning of everything, the dataset is that beginning.

The Dataset - Let's Keep It Simple

Before doing anything in deep learning, we need data. Instead of scraping, cleaning, or searching for a dataset, we'll generate one. Why? Because we want full control to understand every step.

We'll simulate a binary classification problem:

- Class **0**: Non-toxic plants
- Class **1**: Toxic plants

And each plant is represented by two features:

- x_1 : leaf length
- x_2 : leaf width

Let's **create our own tiny world**: 100 plants, each described by two numbers: leaf length and leaf width.

We store the plants in **X**, and we label them in **y**. Let's see the magic code:

```
X, y = make_blobs(n_samples=100, n_features=2, centers=2, random_state=0)
```

See ? Nothing to be afraid of. After this single line, our problem is already fully encoded:

- **X** has shape (100, 2): 100 plants $\times$ 2 features.
- **y** has shape (100, 1) (or (100,)): one label per plant.

This is all a neuron needs: numbers in **X**, answers in **y**. **Its job is to learn the relationship between the two.**

Visualizing the Dataset

```
plt.figure(figsize=(6, 5))
plt.scatter(X[:, 0], X[:, 1], c=y, cmap='viridis', edgecolors='k')
plt.show()
```

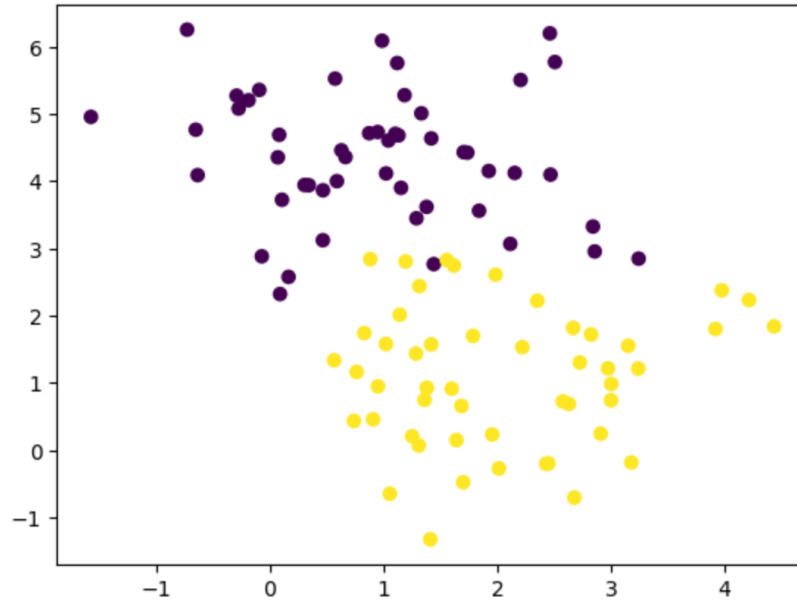


Figure 1: Visual separation of toxic plants (1) in yellow and non-toxic (0) plants in purple.

Pro tip: Always visualize your dataset before modeling. You want to understand how your data is structured and if the problem is linearly separable.

Initialization - The Starting Line

Step 1: Initialize the Model

Every learning algorithm needs a starting point. Before the neuron can learn anything, we need to define what it starts with - that's where parameter initialization comes in.

We define a simple `initialisation` function to set up our model's parameters:

- W is a vector of weights - one per feature (here: x_1, x_2).
- b is the bias - a scalar added after the weighted sum.

Since each plant is described by two features, our neuron only needs two weights and one bias:

$$W \in \mathbb{R}^{2 \times 1}, \quad b \in \mathbb{R}.$$

In code, we simply pick random starting values:

```
def initialisation(X):
    # One weight per feature, plus a single bias
    W = np.random.randn(X.shape[1], 1)
    b = np.random.randn(1)
    return W, b
```

```
W, b = initialisation(X)
```

The exact numbers do not matter at this stage — only their **shape** does. We now have a valid starting point for learning: two weights, one bias, ready to be updated.

Premium tip: Proper dimensions are everything. If your W shape doesn't match your input features, your model won't even start. Always check it before moving forward.

We now have:

$$W = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}, \quad b \in \mathbb{R}$$

Let's move on to the neuron's forward computation.

The Model - Neuron Behavior

Step 2: Process and Activate

This is the heart of the neuron. It receives input, transforms it linearly, and decides whether to activate - not with a brain, but with math.

Let's break it down:

- **Linear transformation:** $Z = X \cdot W + b$
- **Activation:** $A = \sigma(Z) = \frac{1}{1+e^{-Z}}$

The activation function introduces non-linearity. That's what makes the neuron smart - otherwise, it's just linear algebra.

```
def model(X, W, b):  
    Z = np.dot(X, W) + b          # Linear transformation  
    A = 1 / (1 + np.exp(-Z))      # Sigmoid activation  
    return A  
  
A = model(X, W, b)  
print(A.shape)
```

Suppose this returns:

```
(100, 1)
```

Why does this matter? Because **A** must have the same shape as **y**, the ground truth. Only then can we compare the prediction with the actual label for each data point.

The model now outputs a probability for each input:

$$A = \text{sigmoid}(X \cdot W + b)$$

Each value in A is the neuron's confidence that the corresponding plant is toxic.

Next, we'll need a way to evaluate whether these predictions are any good - and that's where the **cost function** comes in.

Cost Function - Log-Loss

Step 3: Measuring the Error

How do we know if our neuron is doing well? We compare its prediction to the truth - and quantify the difference using a cost function.

In binary classification, the most common cost function is the **log-loss**, also called **binary cross-entropy**. It measures how far the predicted probability A is from the actual class label y .

$$\text{Log-loss} = -\frac{1}{m} \sum_{i=1}^m (y_i \cdot \log(a_i) + (1 - y_i) \cdot \log(1 - a_i))$$

In Python:

```
def log_loss(A, y):  
    m = len(y)  
    loss = -1/m * np.sum(y * np.log(A) + (1 - y) * np.log(1 - A))  
    return loss  
  
loss = log_loss(A, y)  
print(loss)
```

What is this value? A single real number that captures the model's *average prediction error*.

- If $\text{loss} \rightarrow 0$: the model predicts very well.
- If $\text{loss} \rightarrow 1$ (or higher): the model is confused and likely making poor predictions.

Our mission during training?

Minimize the log-loss.

Next, we'll need to calculate **how to change** our parameters to reduce that loss - and that's where **gradients** come in.

Gradients - Learning How to Learn

Step 4: Which Way to Go?

To minimize the cost, the model needs to know: *Which direction should I go? And how fast?* That's the job of the **gradient**.

The gradient tells us how the cost changes with each parameter - in other words, how each weight and bias contributes to the prediction error.

We compute the gradients of the log-loss with respect to W and b :

$$\frac{\partial \mathcal{L}}{\partial W}, \quad \frac{\partial \mathcal{L}}{\partial b}$$

In Python, it looks like this:

```
def gradients(A, X, y):
    m = len(y)
    dZ = A - y          # error term
    dW = 1/m * np.dot(X.T, dZ)
    db = 1/m * np.sum(dZ)
    return dW, db

dW, db = gradients(A, X, y)
print("dW_=" + str(dW))
print("db_=" + str(db))
```

- dW is a matrix of the same shape as W - a gradient for each weight.
- db is a scalar - the gradient for the bias.

These gradients point in the direction of maximum error increase. So we'll go the *opposite* way - that's gradient descent.

In the next step, we'll use these gradients to **update the parameters** - and improve our model, step by step.

Parameter Update - Learning by Doing

Step 5: Adjust and Improve

Now that we know *where* the error comes from (thanks to the gradients), let's move our parameters in the right direction - **toward less error**.

This is the heart of the **gradient descent** algorithm. We apply small updates to our weights and bias to reduce the cost, step by step.

Update rules:

$$W = W - \alpha \cdot \frac{\partial \mathcal{L}}{\partial W} \quad b = b - \alpha \cdot \frac{\partial \mathcal{L}}{\partial b}$$

- W is the weight vector
- b is the bias term
- α is the **learning rate** - it controls the step size of each update

The learning rate is crucial:

- If it's too **small**, learning is slow

- If it's too **large**, the model might overshoot and never converge

```
def update(W, b, dW, db, learning_rate=0.1):
    W = W - learning_rate * dW
    b = b - learning_rate * db
    return W, b

W, b = update(W, b, dW, db)
```

Each update moves the model one step closer to the optimal parameters. Train long enough, and the neuron will *learn* how to predict.

Training Loop – Putting It All Together

Step 6: Learn from Experience

Up to now, we built the pieces separately:

- how the neuron **computes** a prediction (model),
- how we **measure** its error (log-loss),
- how we **point** in the right direction (gradients),
- how we **move** W and b (update).

Now we connect everything into a single learning process.

This is where the neuron truly learns: by updating W and b again and again.

In other words, what we did in the previous sections was:

Define the neuron's behaviour: given (x_1, x_2) and (W, b) , it produces a score A , then define how wrong that score is: the log-loss turns A and y into a single number and then define how to fix it: gradients and update tell us how to change W and b .

Alone, each piece is just a formula. **Together, they form a learning loop.**

What we give the neuron:

- X – the input data (leaf length and width),
- y – the true class labels (toxic or not).

What the neuron does with them, at each step:

1. Starts from some initial W and b (initialisation).
2. Computes predictions A for all plants (forward / model).
3. Measures how wrong these predictions are (log-loss).
4. Computes how W and b should change (gradients).

5. Updates W and b slightly (gradient descent).

Then it repeats this cycle many times. Each repetition is one small experience: *“I tried these parameters, I saw the error, I adjusted a bit.”* Over iterations, W and b move toward values that fit the data better. This repetition is called an epoch in this world.

And we will let 100 epochs for this neuron to train.

We can write this learning loop like this :

```
def artificial_neuron_test(X, y, learning_rate=0.1, num_iter=100):
    # 1. Start somewhere: random W and b
    W, b = initialisation(X)

    # Keep track of how the error evolves
    loss = []

    for i in range(num_iter):
        # 2. Forward: make predictions for all plants
        A = model(X, W, b)

        # 3. Measure how wrong they are
        loss.append(log_loss(A, y))

        # 4. Compute how to change W and b
        dW, db = gradients(A, X, y)

        # 5. Take a small step in the right direction
        W, b = update(W, b, dW, db, learning_rate)

    # Visualize the learning process
    plt.plot(loss)
    plt.show()
```

This loop does not “add new magic”. It simply **reuses everything we built** — model, loss, gradients, update — and lets them run together.

At each iteration, the only thing that really changes is:

$$(W, b) \longrightarrow (W', b').$$

That is what “learning” means for a neuron: **slowly turning random W and b into meaningful W and b .**

Run the training:

```
artificial_neuron_test(X, y)
```

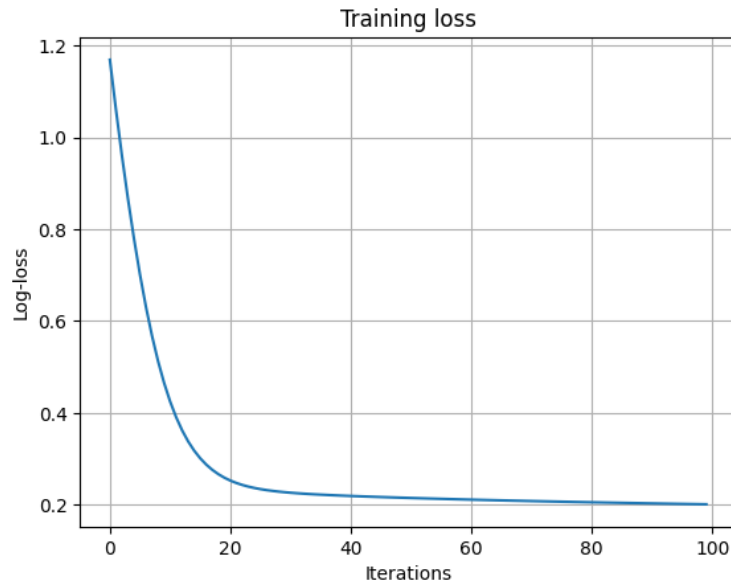


Figure 2: Learning Curve - Log Loss across iterations

This is what learning looks like. The model becomes less wrong over time.

We trained the model for 100 iterations, which means the model had 100 training sessions (or **epochs**) to improve.

From Loss to Accuracy: A Subtle but Important Shift

Up until now, we've been using the **loss function** to evaluate our model - and that's essential. The loss tells us how well the model fits the data during training. It's a continuous value that reflects the quality of the prediction *probabilities*.

But there's a catch: **loss is not accuracy**.

What's the difference?

- The **loss** measures how far the predicted probabilities are from the correct labels. It's sensitive to confidence - predicting 0.51 when the correct class is 1 is penalized less than predicting 0.01.
- The **accuracy** simply measures: *Did we get the class right? Yes or no?*

We round the sigmoid output to 1 if it's ≥ 0.5 , and to 0 otherwise. Then we compare this prediction to the true label.

This is why we now define a new function: `predict()`.

It doesn't return probabilities - it returns the predicted class (0 or 1) for each input.

Once we have that, we can compute the model's **accuracy** by counting how many predictions match the actual labels.

This gives us a simple but powerful insight:

Loss guides the learning. Accuracy tells us if it worked.

In practice: a data scientist trains the neuron model to lower the loss, and then *judges* the model by looking at the accuracy.

In our case, the question is: out of 100 plants, how many were classified correctly?

So, How Do We Predict?

Very simple: the code barely changes. Let's see:

Our model already gives us the output A - a probability between 0 and 1 - using the sigmoid function.

Now we turn that into a final decision:

- If $A \geq 0.5 \Rightarrow$ the plant is **toxic** (Class 1)
- If $A < 0.5 \Rightarrow$ the plant is **non-toxic** (Class 0)

This leads us to define the `predict()` function:

```
def predict(X, W, b):  
    A = model(X, W, b)  
    return A >= 0.5
```

This function transforms a **probability into a decision**.

The closer A is to 0 or 1, the more confident the model is. The closer it is to 0.5, the less certain it becomes.

To compute the overall **accuracy**, we compare the predicted labels to the ground truth, and compute the proportion of correct predictions:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of examples}}$$

This final step closes the loop. You trained a neuron - now it answers.

Let's Test our Trained Neuron on the Data

Now that training is complete, we want to answer a very simple question:

Out of 100 plants, how many did the neuron classify correctly as toxic or non-toxic?

Up to this point, we have focused on the **learning process** itself: the neuron starts from random W and b , makes predictions, measures its error, computes gradients, and updates its parameters. That is what made W and b slowly move in the right direction.

Testing the neuron is not about changing its behaviour. It is about **asking it to answer** — and counting how often it is right.

In practice, nothing fundamental changes in the learning loop:

- We still initialize W and b .
- We still run the same sequence: model $\rightarrow$ log-loss $\rightarrow$ gradients $\rightarrow$ update.
- We still track the loss over iterations.

We simply add, *after training is done*, one more step:

6. Use the final W and b to make predictions on all plants.
7. Compare these predictions to the true labels y and compute the **accuracy**.

Accuracy is just:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of examples}}.$$

Here is how this looks when we put it all together:

```
def artificial_neuron(X, y, learning_rate=0.1, num_iter=100):
    # 1. Initialize parameters
    W, b = initialisation(X)

    # 2. Track how the error evolves over time
    loss = []

    for i in range(num_iter):
        # Forward: predictions for all plants
        A = model(X, W, b)

        # Store current error
        loss.append(log_loss(A, y))

        # Backward: how to change W and b
        dW, db = gradients(A, X, y)

        # Update: take a small step
        W, b = update(W, b, dW, db, learning_rate)

    # 3. Training is over: W and b are now "trained"

    # Use the final parameters to make class predictions
    y_pred = predict(X, W, b)

    # Measure how often the neuron is correct
    accuracy_score(y, y_pred)

    # Visualize the loss over iterations
    plt.plot(loss)

    return W, b
```

```
W, b = artificial_neuron(X, y)
```

You might obtain an accuracy score like 0.92 or 0.86, meaning:

The neuron correctly predicted whether a plant was toxic or not in 92% (or 86%) of the cases.

And remember: this decision is made using only two numbers per plant — leaf length and leaf width.

By the end of training, you also recover the final, optimized parameters W and b : the precise settings the neuron has settled on through gradient descent, after seeing the data many times.

If the loss curve steadily decreases and then stabilizes, and the accuracy is reasonably high, **you have successfully trained your first artificial neuron.**

At this point, everything you built before comes together:

- the **model** turns (X, W, b) into predictions A ,
- the **loss** turns (A, y) into a single number,
- the **gradients** tell you how to change W and b ,
- the **updates** slowly refine these parameters,
- and finally, the **accuracy** tells you how well the whole story worked.

Learning, here, is nothing more — and nothing less — than **turning random W and b into W and b that get most answers right.**

Predicting a New Sample

So far, we have only evaluated the neuron on the plants it has already seen during training. But the real question is more interesting:

What happens when we show it a plant it has never seen before?

Let's imagine a new plant, freshly measured:

- leaf length = 2
- leaf width = 1

We place this new plant in the same 2D space as our training data:

```
# A new plant: [leaf length, leaf width]
new_plant = np.array([[2.0, 1.0]]) # shape (1, 2)

plt.scatter(X[:, 0], X[:, 1], c=y, cmap='viridis')
plt.scatter(new_plant[0, 0], new_plant[0, 1], c='r')
plt.title("New plant among known data points")
plt.xlabel("Leaf length")
plt.ylabel("Leaf width")
plt.grid(True)
plt.show()
```

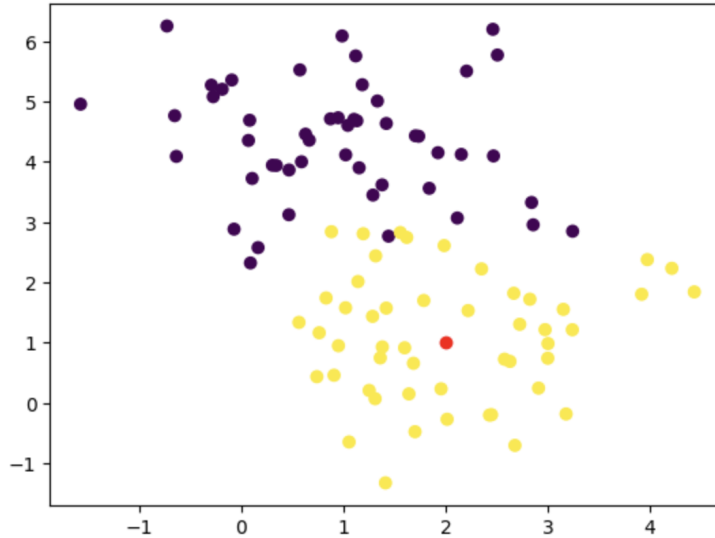


Figure 3: New plant (red) among known data points

The red point represents a new plant whose leaf length and width have just been measured. It lives in the exact same feature space as our training plants.

Now the question becomes very concrete: **What does our trained neuron think about this new plant?**

We will ask the neuron two things:

1. the **probability** that the plant is toxic;
2. the final **decision** (toxic or non-toxic) based on that probability.

```
def predict_plant(X, W, b):
    A = model(X, W, b)
    print(A)
    return A >= 0.5

predict_plant(new_plant, W, b)
```

Suppose the neuron outputs a probability of **0.93**. Since $0.93 \geq 0.5$, the predicted label is 1:

According to the model, this new plant is toxic (class 1), and it is about 93% confident in that decision.

This reveals a key idea: the neuron’s output is not just a hard “yes/no” answer. It is a **probability** — a degree of certainty.

- Probabilities close to 1 or 0 mean **high confidence**.
- Probabilities close to 0.5 mean **uncertainty**: the neuron is hesitating.

If we were to look at all the points where the neuron outputs exactly 0.5, we would obtain a kind of frontier: the place where it is **exactly undecided**.

This frontier is called the **decision boundary**: the line that separates “toxic” from “non-toxic” according to the model.

Let's Plot the Decision Boundary

From the definition of the sigmoid, we know that:

$$a = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

When $a = 0.5$, the neuron is exactly undecided. Solving $\sigma(z) = 0.5$ gives $z = 0$.

So the set of points where the neuron **hesitates** (probability = 0.5) is precisely the set of points where:

$$z = Wx + b = 0.$$

In 2D, this equation describes a straight line. This line is the **decision boundary**.

A Short Note Before the Maths

How do we find the exact equation of this decision line?

In this guide, nothing is left in the dark — if you want the maths, you'll find them just below. If you prefer to skip and jump straight to the plot, you absolutely can.

Math Corner: From $Wx + b = 0$ to the Line Equation

For the decision boundary, we said that the neuron is exactly undecided when

$$a = \sigma(z) = 0.5 \quad \Longleftrightarrow \quad z = 0.$$

And we know that

$$z = Wx + b.$$

In our 2D setting, this becomes more concrete. Let us write:

$$W = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}, \quad x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

Then

$$z = Wx + b = w_1x_1 + w_2x_2 + b.$$

The decision boundary is defined by the set of points where the neuron is undecided:

$$w_1x_1 + w_2x_2 + b = 0.$$

This is an equation with two variables, x_1 and x_2 . If we want to **draw** this curve in the (x_1, x_2) plane, it is convenient to express x_2 as a function of x_1 .

Assuming $w_2 \neq 0$, we can rearrange the equation:

$$w_1x_1 + w_2x_2 + b = 0 \quad \implies \quad w_2x_2 = -w_1x_1 - b \quad \implies \quad x_2 = -\frac{w_1}{w_2}x_1 - \frac{b}{w_2}.$$

This is exactly the equation of a straight line:

$$x_2 = mx_1 + c,$$

with

$$m = -\frac{w_1}{w_2} \quad \text{and} \quad c = -\frac{b}{w_2}.$$

END OF MATH

So, we draw the decision boundary:

- we choose a range of x_1 values (leaf lengths),
- we compute the corresponding x_2 values (leaf widths) using the formula above,
- and we plot all the pairs (x_1, x_2) as a line.

In code, this is exactly what we do. Let's draw that line on top of our data and highlight the new plant:

```
# x1: range of leaf lengths we want to cover
x1 = np.linspace(-1, 4, 100)

# For each x1, solve w1 * x1 + w2 * x2 + b = 0 -> x2 = (-w1 * x1 - b) / w2
x2 = (-W[0] * x1 - b) / W[1]
```

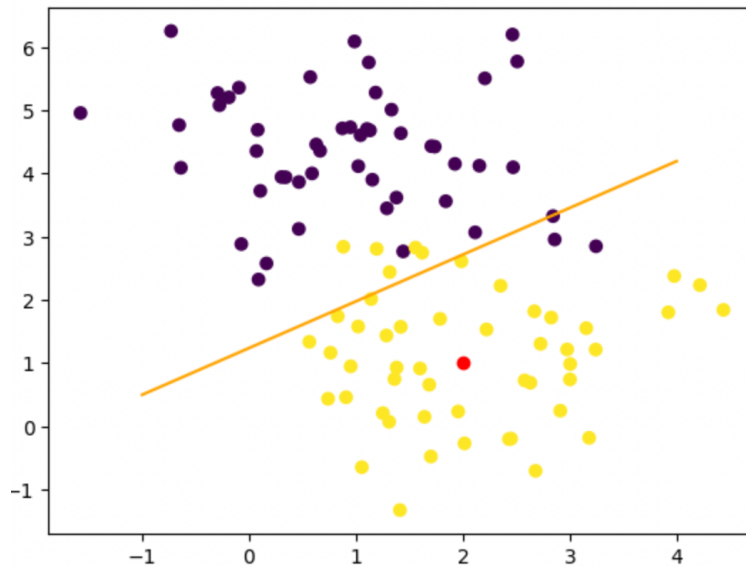


Figure 4: The decision boundary separates toxic and non-toxic classes.

What do we see?

- The **orange line** is the decision boundary: all points on this line give a probability of 0.5.
 - One side of the line is classified as **toxic** (class 1), the other as **non-toxic** (class 0).
-
- The **red point** (our new plant) lies clearly on one side of the line, and far from it. This matches the high confidence we saw earlier (probability close to 1).
 - Points that sit **close to the boundary** are much more ambiguous: for them, the neuron's output will be near 0.5, and small changes in W , b or the input could flip the decision.

Why is this the “zone of doubt”? Because when $z \approx 0$, the sigmoid output satisfies $a \approx 0.5$ – and the neuron is **maximally unsure** about its decision.

At the decision boundary, it’s almost like flipping a coin. A tiny change can send the prediction to one class or the other, and misclassifications are more likely there.

Even if we train the model for many epochs, this type of neuron — a **linear classifier** — has a built-in limitation:

It can only learn linear boundaries.

It can draw one straight line to separate the classes. When the true frontier between classes is curved or highly complex, this simple neuron will struggle — and that is exactly why we will later need **multiple neurons** and **layers**: to bend and shape more complex decision boundaries. So while it performs well in many cases, it naturally struggles when the true boundary between classes is curved, tangled, or deeply nonlinear.

Seeing the Neuron’s Confidence in 3D

So far, we have looked at the decision in two ways:

- point by point — a new plant and its predicted probability,
- globally — the decision boundary line where the neuron is exactly undecided.

Now, instead of reading equations or code, **let’s look at what the neuron has learned visually**: how its “intelligence” is shaped over the whole input space.

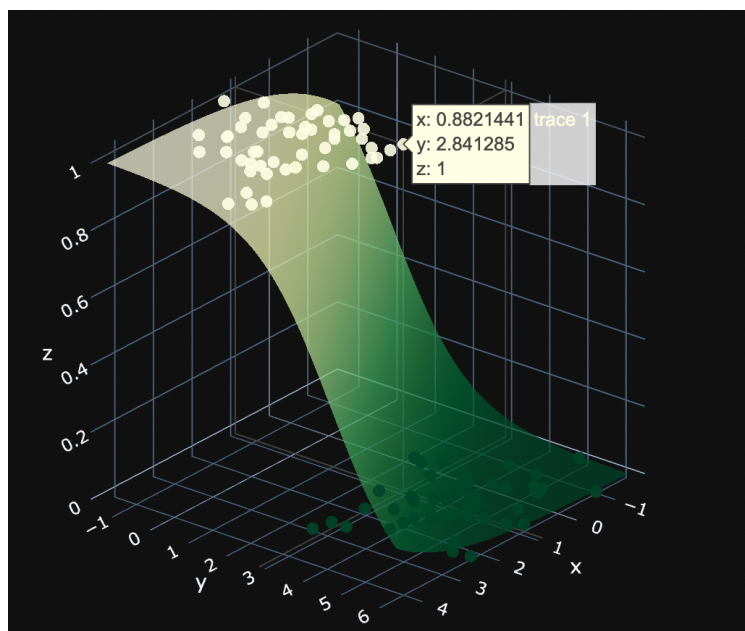


Figure 5: Points near the decision boundary are the most ambiguous.

This figure gives another view of the decision-making process in 3D, using the **sigmoid function** as our activation.

Each point in the plane (leaf length, leaf width) is colored according to the neuron's output: the predicted probability that the plant is toxic.

From the plot, you can read the neuron's behaviour at a glance:

- The sigmoid outputs values between 0 and 1 — a probability-like score.
- If the output is close to **1**, the model predicts the plant is **toxic** (class 1).
- If the output is close to **0**, the model predicts the plant is **non-toxic** (class 0).

The **center of the sigmoid** — where outputs are near 0.5 — is where predictions are the most fragile. Geometrically, this region hugs the decision boundary we just plotted: many points lying close to that line fall in this ambiguous zone, and some of them are **very likely misclassified**.

This is why the sigmoid is so useful in binary classification: it takes the linear score $z = Wx + b$ and turns it into a smooth probability between 0 and 1, with a natural “uncertainty band” around the decision boundary.

So congrats - you just understood a binary classification model.

Going Beyond 2D: Higher-Dimensional Data

In real life, we rarely have just two features. A single example can have **dozens** or **thousands** of inputs (pixels, sensors, words, etc.), so thousands of dimensions. Beyond 2D, we can no longer draw neat scatter plots or decision lines — but the learning loop you built stays exactly the same.

How do we “see” in high dimensions?

We rely on metrics instead of pictures:

- the **learning curve** (loss over time),
- the **accuracy** (how often we are right).

These become our new eyes when geometry is no longer visible.

In the next notebook, we will reuse this same neuron — same equations — and let it tackle a more realistic task: classifying images of **cats and dogs**.